

Tham chiếu

Chuẩn bị

```
create table departments (  
    id int primary key,  
    name varchar(255)  
);  
  
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_name VARCHAR(100) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(id)  
        ON DELETE RESTRICT  
        ON UPDATE RESTRICT  
);  
  
INSERT INTO departments (id, name) VALUES  
(4, 'it'),  
(1, 'HR'),  
(2, 'Engineering'),  
(3, 'Marketing');  
  
INSERT INTO employees (emp_name, dept_id) VALUES  
( 'Alice', 1),  
( 'Bob', 2),  
( 'Charlie', 2),  
( 'Daisy', 3);  
  
DELETE FROM departments WHERE id = 4;
```

Hành động khi tham chiếu

- ❖ Các hành động để quyết định cách xử lý khi bản ghi trong bảng cha (bảng được tham chiếu) khi bị xóa hoặc cập nhật.
 - **CASCADE** – Tự động xóa hoặc cập nhật các bản ghi con.
 - **SET NULL** – Đặt giá trị của khóa ngoại thành NULL khi bản ghi cha bị xóa hoặc cập nhật.
 - **SET DEFAULT** – Đặt giá trị của khóa ngoại về một giá trị mặc định khi bản ghi cha bị xóa hoặc cập nhật (ít được MySQL hỗ trợ).
 - **RESTRICT** – Chặn xóa/cập nhật nếu có bản ghi con liên kết.
 - **NO ACTION** – Tương tự RESTRICT, nhưng kiểm tra khi giao dịch kết thúc (ít phổ biến hơn).

CASCADE – Tự động cập nhật hoặc xóa

- ❖ Nếu muốn tự động xóa hoặc cập nhật dữ liệu con.
- ❖ ON DELETE CASCADE: Khi một phòng ban bị xóa, tất cả nhân viên thuộc phòng đó cũng bị xóa.
- ❖ ON UPDATE CASCADE: Nếu dept_id của một phòng ban thay đổi, tất cả nhân viên thuộc phòng đó cũng sẽ được cập nhật

CASCADE – Tự động cập nhật hoặc xóa

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_name VARCHAR(100) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
        ON DELETE CASCADE  
        ON UPDATE CASCADE  
);
```

```
DELETE FROM departments WHERE dept_id = 1;  
-- → Tất cả nhân viên có dept_id = 1 sẽ bị xóa theo.
```

```
UPDATE departments SET dept_id = 10 WHERE dept_id = 2;  
-- → Tất cả nhân viên có dept_id = 2 sẽ đổi thành  
dept_id = 10.
```

SET NULL – Đặt khóa ngoại thành NULL

- ❖ Nếu muốn giữ dữ liệu con nhưng ngắt liên kết
- ❖ ON DELETE SET NULL: Khi một phòng ban bị xóa, dept_id của nhân viên sẽ trở thành NULL (thay vì xóa nhân viên).
- ❖ ON UPDATE SET NULL: Khi dept_id bị thay đổi, tất cả nhân viên có dept_id đó sẽ được đặt NULL

SET NULL – Đặt khóa ngoại thành NULL

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_name VARCHAR(100) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
        ON DELETE SET NULL  
        ON UPDATE SET NULL  
);
```

```
DELETE FROM departments WHERE dept_id = 1;  
-- → Các nhân viên có dept_id = 1 sẽ có dept_id = NULL  
KHÔNG bị xóa.
```


RESTRICT – Chặn xóa hoặc cập nhật

- ❖ Nếu không muốn xóa dữ liệu khi có liên kết
- ❖ ON DELETE RESTRICT: Không cho phép xóa departments nếu có nhân viên thuộc phòng đó.
- ❖ ON UPDATE RESTRICT: Không cho phép cập nhật dept_id nếu có nhân viên liên kết.

RESTRICT – Chặn xóa hoặc cập nhật

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_name VARCHAR(100) NOT NULL,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
        ON DELETE RESTRICT  
        ON UPDATE RESTRICT  
);
```

```
DELETE FROM departments WHERE dept_id = 1;
```

-- ❌ Lỗi: Không thể xóa vì có nhân viên thuộc phòng này.

```
UPDATE departments SET dept_id = 10 WHERE dept_id = 2;
```

-- ❌ Lỗi: Không thể cập nhật vì có nhân viên thuộc phòng này.

Use case sử dụng RESTRICT, SET NULL, CASCADE

1. Đơn hàng (orders) và Sản phẩm. Nếu thông tin sản phẩm bị xóa, ...
2. Khách sạn (hotels) và Phòng (rooms). Nếu thông tin khách sạn bị xóa, ...
3. Trong thông tin nhân viên, có thông tin của manger_id. Nếu thông tin của nhân viên trưởng phòng bị xóa, ...

Use case sử dụng RESTRICT, SET NULL, CASCADE

1. Đơn hàng (orders) và Sản phẩm. Nếu thông tin sản phẩm bị xóa, ...

⇒ RESTRICT. Ngăn chặn việc xóa sản phẩm khi đang thuộc một đơn hàng nào đó

2. Khách sạn (hotels) và Phòng (rooms). Nếu thông tin khách sạn bị xóa, ...

⇒ CASCADE. Khi xóa thông tin khách sạn, sẽ xóa các phòng thuộc khách sạn đó

3. Trong thông tin nhân viên, có thông tin của manger_id. Nếu thông tin của nhân viên trưởng phòng bị xóa, ...

⇒ SET NULL. Khi trưởng phòng bị xóa, nhân viên đó sẽ đặt manager_id = NULL

Soft Delete

PHỤ THUỘC HÀM

Mục đích của Chuẩn hóa và Phụ thuộc hàm

- ❖ **Chuẩn hóa** là một kỹ thuật tạo ra một tập các quan hệ với các thuộc tính từ các yêu cầu cho trước về dữ liệu cần mô hình hóa của tổ chức.
- ❖ Quá trình chuẩn hóa đầu tiên được phát triển bởi Codd vào năm 1972.
- ❖ Việc chuẩn hóa thường được thực hiện như một chuỗi các kiểm tra trên một quan hệ để xác định xem nó có thỏa mãn hay vi phạm các yêu cầu của một dạng chuẩn cho trước nào đó hay không.

Mục đích của Chuẩn hóa và Phụ thuộc hàm

- ❖ Codd định nghĩa 3 dạng chuẩn: dạng chuẩn 1 (1NF), dạng chuẩn 2 (2NF), và dạng chuẩn 3 (3NF).
- ❖ Năm 1974, Boyce và Codd cùng giới thiệu một dạng chuẩn mạnh hơn 3NF được gọi là chuẩn Boyce-Codd (BCNF).
- ❖ Cả 4 dạng chuẩn đều dựa trên sự **phụ thuộc hàm giữa các thuộc tính trong một quan hệ**.
- ❖ **Một phụ thuộc hàm** mô tả mối quan hệ giữa các thuộc tính trong một quan hệ.

Mục đích của Chuẩn hóa và Phụ thuộc hàm

- ❖ Năm 1977, 1979 và các năm tiếp theo, người ta giới thiệu dạng chuẩn 4 (4NF), dạng chuẩn 5 (5NF) và các dạng chuẩn mức cao hơn.
 - Tuy nhiên, rất hiếm khi gặp.
- ❖ **Chuẩn hóa** giúp người thiết kế kiểm tra và chuyển các quan hệ về một dạng chuẩn hóa cụ thể nào đó nhằm ngăn chặn các dị thường khi thực hiện các thao tác cập nhật thông tin.

Mục đích của Chuẩn hóa và Phụ thuộc hàm

- ❖ Mục đích của việc thiết kế CSDL quan hệ: nhóm các thuộc tính vào thành các quan hệ sao cho tối thiểu hóa sự dư thừa dữ liệu => giảm không gian lưu trữ và tránh dị thường thông tin khi cập nhật dữ liệu.
- ❖ Xét ví dụ lược đồ quan hệ staffbranch.

staffbranch

<u>staff#</u>	sname	position	salary	branch#	baddress
SL21	Kristy	manager	30000	B005	22 Deer Road
SG37	Debi	assistant	12000	B003	163 Main Street
SG14	Alan	supervisor	18000	B003	163 Main Street
SA9	Traci	assistant	12000	B007	375 Fox Avenue
SG5	David	manager	24000	B003	163 Main Street
SL41	Anna	assistant	10000	B005	22 Deer Road

Mục đích của Chuẩn hóa và Phụ thuộc hàm

- ❖ Quan hệ *staffbranch* có dư thừa dữ liệu. Thông tin chi tiết của một chi nhánh ngân hàng (branch) sẽ bị lặp cho mỗi nhân viên (staff) làm việc tại chi nhánh đó.
- ❖ Nếu tách riêng quan hệ staff và branch thì thông tin về từng chi nhánh ngân hàng chỉ xuất hiện duy nhất một lần.

staff

<u>staff#</u>	sname	position	salary	branch#
SL21	Kristy	manager	30000	B005
SG37	Debi	assistant	12000	B003
SG14	Alan	supervisor	18000	B003
SA9	Traci	assistant	12000	B007
SG5	David	manager	24000	B003
SL41	Anna	assistant	10000	B005

branch

<u>branch#</u>	baddress
B005	22 Deer Road
B003	163 Main Street
B007	375 Fox Avenue

DƯ THỪA VÀ DỊ THƯỜNG

❖ Khi thực hiện chèn thêm dữ liệu.

- Để chèn thêm thông tin chi tiết cho các nhân viên mới vào staffbranch, cần phải thêm thông tin chi tiết về chi nhánh tương ứng cho mỗi bản ghi nhân viên.
- Để chèn thêm thông tin cho một branch mới mà hiện tại chưa có nhân viên nào, cần chèn thêm các giá trị Null cho các thuộc tính về nhân viên, như staff#, sname, ...

DƯ THỪA VÀ DỊ THƯỜNG

❖ Khi thực hiện xóa dữ liệu.

- Giả sử trong staffbranch có một chi nhánh chỉ còn một nhân viên cuối cùng. Nếu xóa thông tin về nhân viên này thì các thông tin về chi nhánh đó cũng sẽ bị xóa khỏi CSDL.

DƯ THỪA VÀ DỊ THƯỜNG

❖ Khi thực hiện cập nhật dữ liệu.

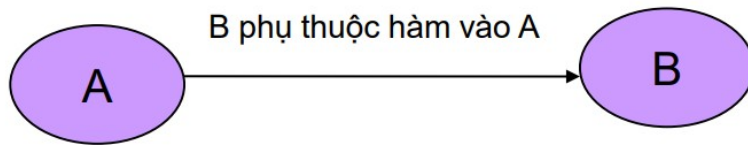
- Giả sử muốn thay đổi giá trị của một trong các thuộc tính của một chi nhánh nào đó trong quan hệ staffbranch, ví dụ: địa chỉ chi nhánh, thì cần phải cập nhật tất cả các bộ của các nhân viên làm việc tại chi nhánh đó..

DƯ THỪA VÀ DỊ THƯỜNG

- ❖ 2 tính chất quan trọng khi tách một lược đồ quan hệ thành một tập các lược đồ nhỏ hơn
 - **Kết nối không tổn thất thông tin**: đảm bảo mọi thể hiện của quan hệ ban đầu có thể xác định được từ các thể hiện liên quan trong các quan hệ nhỏ hơn.
 - **Bảo toàn sự phụ thuộc hàm**: đảm bảo một ràng buộc trên quan hệ ban đầu có thể được duy trì bằng cách sử dụng đơn giản một số ràng buộc trên mỗi quan hệ nhỏ hơn.

Phụ thuộc hàm

- ❖ **Một phụ thuộc hàm** thể hiện ngữ nghĩa của các thuộc tính trong một quan hệ: một thuộc tính có quan hệ với thuộc tính khác như thế nào và xác định các phụ thuộc hàm giữa các thuộc tính đó.
- ❖ Phụ thuộc hàm là ràng buộc giữa các thuộc tính.
- ❖ Xét một quan hệ với các thuộc tính A và B, với thuộc tính B phụ thuộc hàm vào thuộc tính A. **Nếu biết giá trị của A sẽ tìm thấy một giá trị duy nhất của B.** Nhưng với mỗi giá trị của B cho trước có thể có *nhiều giá trị khác nhau* tương ứng của A



Phụ thuộc hàm

- ❖ Phụ thuộc hàm từ X vào Y được ký hiệu là $X \rightarrow Y$, với X là vế trái và Y là vế phải của phụ thuộc hàm
- ❖ Có thể diễn đạt cách khác: Y phụ thuộc hàm vào X hoặc X xác định hàm Y
- ❖ Một phụ thuộc hàm là một tính chất của lược đồ quan hệ R và không phải là tính chất của trạng thái quan hệ $r(R)$
- ❖ Phụ thuộc hàm $X \rightarrow Y$ hiển nhiên nếu $X \supseteq Y$
- ❖ Nếu X là 1 siêu khóa của R $\{A_1, A_2, \dots, A_n\}$, thì $X \rightarrow \{A_1, A_2, \dots, A_n\}$

Xác định các Phụ thuộc hàm

❖ Xét quan hệ `staff_branch`

Để xác định các phụ thuộc hàm, cần hiểu rõ ngữ nghĩa của các thuộc tính khác nhau trong mỗi lược đồ quan hệ.

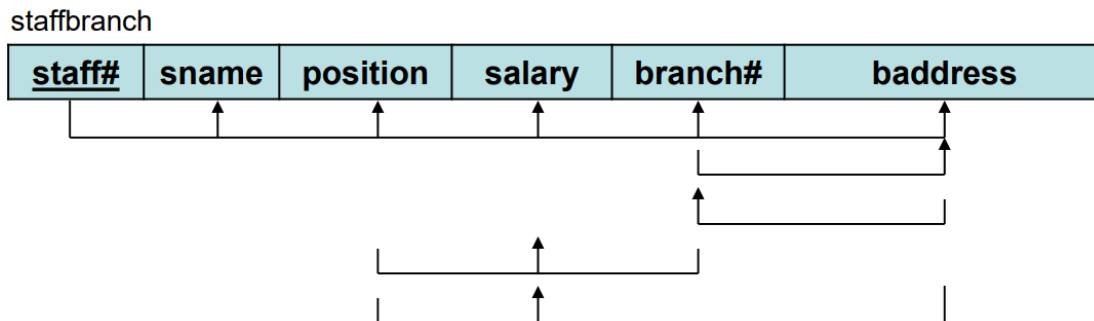
Ví dụ: **Vị trí của mỗi nhân viên** và **chi nhánh** sẽ xác định lương của họ.

❖ Cần hiểu rõ về tổ chức. Đây là nhiệm vụ của giai đoạn phân tích yêu cầu bài toán và giai đoạn thiết kế mức khái niệm.

Xác định các Phụ thuộc hàm

❖ Các phụ thuộc hàm được xác định như sau

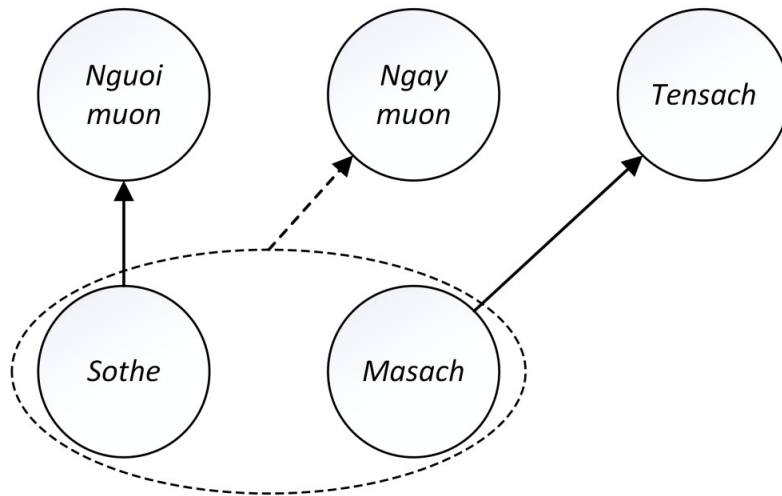
- staff# → sname, position, salary, branch#, baddress
- branch# → baddress
- baddress → branch#
- branch#, position → salary
- baddress, position → salary



Phụ thuộc hàm - Ví dụ 1

❖ MUONSACH(Sothe, MaSach, Nguoimuon, Tensach, Ngaymuon) có các PTH:

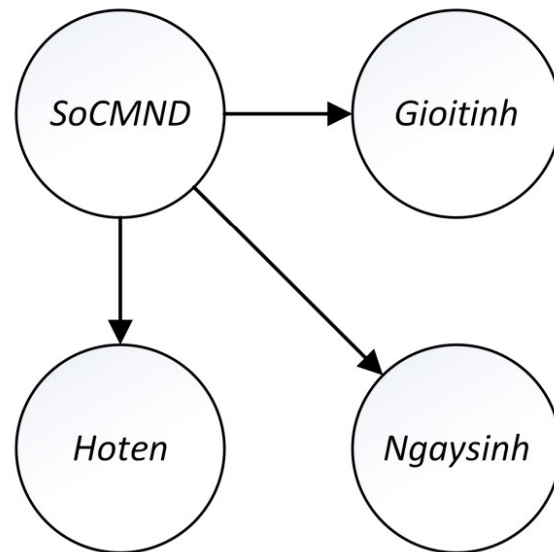
- Sothe → Nguoimuon
- Masach → Tensach
- Sothe, Masach → Ngaymuon



Phụ thuộc hàm - Ví dụ 2

❖ CONGDAN(SoCMND, Hoten, Ngaysinh, Gioitinh) có các phụ thuộc hàm:

- SoCMND $\rightarrow$ Hoten
- SoCMND $\rightarrow$ Ngaysinh
- SoCMND $\rightarrow$ Gioitinh



Phụ thuộc hàm hiển nhiên

- ❖ Trong quá trình xác định các phụ thuộc hàm, **cần bỏ qua các phụ thuộc hàm hiển nhiên đúng**.
- ❖ Phụ thuộc hàm $X \rightarrow Y$ hiển nhiên đúng nếu $X \supseteq Y$
 - { staff#, sname } $\rightarrow$ sname
 - { staff#, sname } $\rightarrow$ staff#
- ❖ Các phụ thuộc hàm này không cung cấp thêm các thông tin cần thiết nào về các ràng buộc toàn vẹn đối với quan hệ.

Phụ thuộc hàm suy diễn được

- ❖ Giả sử F là một tập phụ thuộc hàm trên lược đồ quan hệ R
- ❖ $X \rightarrow Y$ suy diễn được từ tập phụ thuộc hàm F , được ký hiệu là $F \models X \rightarrow Y$
- ❖ Tập tất cả các phụ thuộc hàm được suy diễn từ một tập phụ thuộc hàm F được gọi là **bao đóng** của F và được ký hiệu là F^+ .
- ❖ **Bao đóng** của tập phụ thuộc hàm F , được định nghĩa như sau:

$$F^+ = F \cup \{X \rightarrow Y \mid F \models X \rightarrow Y\}$$

Các luật suy diễn cho các phụ thuộc hàm

- ❖ Một tập các luật suy diễn là cần thiết để suy diễn ra tập các phụ thuộc hàm dựa vào F. Sáu luật suy diễn được biết đến nhiều nhất được áp dụng cho các phụ thuộc hàm như sau:

- IR1: Luật phản xạ: nếu $X \supseteq Y$, thì $X \rightarrow Y$
- IR2: Luật tăng trưởng: nếu $X \rightarrow Y$, thì $XZ \rightarrow YZ$
- IR3: Luật bắc cầu: nếu $X \rightarrow Y$ và $Y \rightarrow Z$, thì $X \rightarrow Z$
- IR4: Luật chiếu: Nếu $X \rightarrow YZ$, thì $X \rightarrow Y$ và $X \rightarrow Z$
- IR5: Luật cộng thêm: Nếu $X \rightarrow Y$ và $X \rightarrow Z$, thì $X \rightarrow YZ$
- IR6: Luật giả bắc cầu: Nếu $X \rightarrow Y$ và $YZ \rightarrow W$, thì $XZ \rightarrow W$

3 Tiên đề
Armstrong

Các luật suy diễn cho các phụ thuộc hàm

Hệ tiên đề Amstrong đóng vai trò là một tập luật **cần thiết và đầy đủ** cho việc tạo ra bao đóng của một tập các phụ thuộc hàm.



Ví dụ

- ❖ Cho lược đồ quan hệ: $R = (A, B, C, D, E, F, G, H, I, J)$ và
- ❖ $F = \{AB \rightarrow E, AG \rightarrow J, BE \rightarrow I, E \rightarrow G, GI \rightarrow H\}$

a. Hỏi $F \models AB \rightarrow GH$?

b. Hỏi $F \models BE \rightarrow H$?

Ví dụ

- ❖ Cho lược đồ quan hệ: $R = (A, B, C, D, E, F, G, H, I, J)$ và
- ❖ $F = \{AB \rightarrow E, AG \rightarrow J, BE \rightarrow I, E \rightarrow G, GI \rightarrow H\}$

a. Hỏi $F \models AB \rightarrow GH$?

1. $AB \rightarrow E$, đã cho trong F
2. $AB \rightarrow B$, luật phản xạ IR1
3. $AB \rightarrow BE$, luật cộng thêm IR5 từ bước 1 và 2
4. $BE \rightarrow I$, đã cho trong F
5. $AB \rightarrow I$, luật bắc cầu IR3 từ bước 3 và 4
6. $E \rightarrow G$, đã cho trong F

b. Hỏi $F \models BE \rightarrow H$?

7. $AB \rightarrow G$, luật bắc cầu IR3 từ bước 1 và 6
 8. $AB \rightarrow GI$, luật cộng thêm IR5 từ bước 5 và 7
 9. $GI \rightarrow H$, đã cho trong F
 10. $AB \rightarrow H$, luật bắc cầu IR3 từ bước 8 và 9
 11. $AB \rightarrow GH$, luật cộng thêm IR5 từ bước 7 và 10
- => kết quả được chứng minh.

Các luật suy diễn cho các phụ thuộc hàm

Ngoài các phụ thuộc hàm hiển nhiên, còn có nhiều các **phụ thuộc hàm khác** cũng thỏa mãn với các thể hiện của quan hệ mà đã thỏa mãn các phụ thuộc hàm trong F .

